

Sistemas Operativos

Facultad de Informática
Universidad Complutense de Madrid

Módulo 3 - Planificación de procesos



Outline

1 Introducción

2 Algoritmos clásicos de planificación

3 Planificación SMP

4 Planificación en Linux

Niveles de planificación

Los planificadores se clasifican tradicionalmente en tres tipos:

- Planificador a largo plazo. Controla la admisión de trabajos en el sistema (qué procesos pueden ejecutarse)
- Planificador a medio plazo. Mueve procesos entre la memoria principal y el almacenamiento secundario (swapping de procesos)
- Planificador a corto plazo. Decide cuál de los procesos preparados se ejecutará a continuación en la CPU

Planificadores en SO

- El único planificador presente en los SO de propósito general actuales es el planificador a corto plazo.
- Los planificadores a largo plazo se implementan mediante aplicaciones dedicadas de gestión de trabajos (p. ej., Slurm o PBS).
- Los planificadores a medio plazo han sido sustituidos por mecanismos de paginación. Los SO móviles (p. ej., Android) pueden en su lugar suspender aplicaciones en segundo plano.

Planificación

- Objetivos
  - Optimizar uso de las CPUs
  - Minimizar tiempo de espera
  - Ofrecer reparto equitativo (justicia)
  - Proporcionar grados de urgencia (prioridades)

Tipos de algoritmos de planificación

- **No expropiativo**: el proceso conserva la CPU hasta que (1) se bloquea, (2) la cede expresamente o (3) termina su ejecución.
- **Expropiativo**: el SO puede expulsar al proceso de la CPU
  - Exige un reloj que interrumpe periódicamente

Planificación y estados de proceso

- Las decisiones de planificación están ligadas al modelo de transición de estados de una tarea:
  - Una tarea entra en la cola de preparados tras su creación (`clone()`) o tras ser despertada (`wake_up()`) de una espera
  - `schedule()` elige una tarea preparada y la pasa a `TASK_RUNNING` en la CPU
  - Mientras se ejecuta, una tarea puede dejar la CPU de cinco formas:
    1. Se bloquea esperando un evento (`wait_event()`) y pasa a `TASK_INTERRUPTIBLE`/ `TASK_UNINTERRUPTIBLE`
    2. Es desalojada por el planificador (apropiación) y vuelve a la cola de preparados
    3. Cede voluntariamente la CPU (`sched_yield()`) y vuelve a la cola de preparados
    4. Termina (`exit()`) y pasa a `ZOMBIE`
    5. Recibe una señal (`signal()`) que fuerza su terminación

Planificación y estados de proceso

- Expropiativo frente a no expropiativo, revisitado
  - **No expropiativo**: solo las transiciones a bloqueado (espera) o terminado sacan al proceso de la CPU
  - **Expropiativo**: el planificador puede además mover una tarea en ejecución directamente de vuelta a preparado

Estructuras de datos del planificador

Estructuras de datos

- El planificador mantiene los procesos/hilos en una cola (*run queue*)
  - Típicamente se implementa como lista doblemente enlazada
- La *run queue* está formada por los BCPs de los procesos listos para ejecutar
  - El proceso que está actualmente en ejecución en la CPU no se mantiene en la *run queue*
  - El planificador no gestiona procesos en estado "bloqueado"
- Algunos algoritmos de planificación mantienen varias colas de procesos
  - Por prioridad, por tipo, ...

Activación del planificador

Puntos de activación

- Periódicamente (interrupción del temporizador de la CPU)
- Como resultado del procesamiento de alguna interrupción generada por otros dispositivos de E/S
- El proceso en ejecución causa una excepción que lo bloquea (fallo de página) o fuerza su terminación (violación de segmento)
- Cuando el proceso en ejecución termina
- El proceso realiza una llamada bloqueante
- Cesión voluntaria del procesador
  - `sched_yield()`
- Se desbloquea un proceso más "importante" que el actual
  - *expropiación de usuario*

Planificación sin tick (tickless)

Activación periódica

- El temporizador hardware lanza una interrupción a intervalos regulares
- El intervalo define la *tasa de tick* (p. ej., Linux 100 Hz por defecto, CONFIG*HZ)
- El manejador de la interrupción dispara el evento de planificación

Activación sin tick

- La interrupción del temporizador se programa en su lugar bajo demanda, justo antes del siguiente evento que realmente necesita atención
- Mejor perfil de ahorro energético para sistemas embebidos
- Linux usa un modelo híbrido sin tick: *adaptativo* cuando solo hay una tarea ejecutable, e *idle* (ahorro energético máximo) cuando la CPU no tiene ninguna tarea ejecutable

Cambio de contexto (I)

Cambio de contexto

- Conjunto de acciones que realiza el SO para cambiar el proceso que se ejecuta en una CPU:
  - Cambiar el contexto de memoria virtual
    - Actualizar los registros de gestión de memoria al contexto de memoria del nuevo proceso (`mm_struct` , p. ej., dirección de la tabla de páginas, cachés asociadas...)
  - Cambiar el contexto de ejecución de la CPU (`thread_struct`)
    - Guardar los registros de propósito general, el contador de programa, el puntero de pila
    - Guardar los registros específicos de la arquitectura (p. ej., el estado de la FPU)
    - Actualizar el puntero de pila a la nueva tarea
    - Cargar los registros de la nueva tarea
    - Reanudar la ejecución de la nueva tarea (en modo usuario)

Cambio de contexto (II)

Coste

- Un cambio de contexto completo de proceso es una operación costosa
- Un cambio de contexto entre hilos del mismo proceso es más barato (no hay cambio de memoria virtual), pero sigue sin ser despreciable

Métricas del planificador

Métricas por entidad (proceso o thread)

- Tiempo de ejecución o de retorno
  - $T_{ejecucin} = T_{fin} - T_{creacin}$
- Tiempo de espera: tiempo total que el proceso pasa esperando en la cola del planificador (listo para ejecutar)
- Tiempo de respuesta:
  - $T_{respuesta} = T_{primerUsoDeCPU} - T_{creacin}$

Métricas globales

- Porcentaje de utilización del procesador
- Productividad: número de trabajos completados por unidad de tiempo

Algoritmos no expropiativos

- El planificador no quita la CPU al proceso una vez que está en ejecución, a no ser esta la ceda voluntariamente, termine o se bloquee por E/S

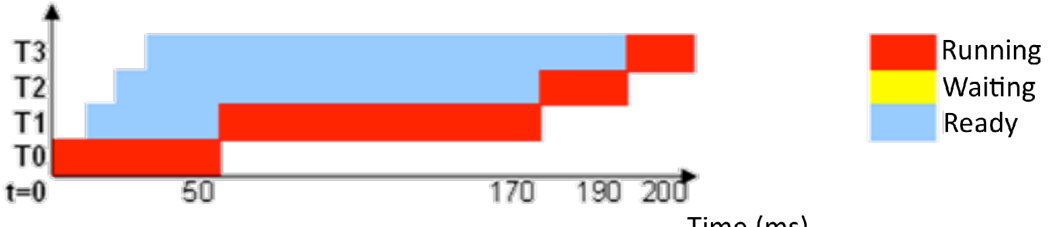
Algoritmos

- Primero en llegar primero en ejecutar o FCFS (*First-Come First-Served*)
- Primero el trabajo más corto o SJF (*Shortest Job First*)
  - También conocido como SPN (*Shortest Process Next*)
- Planificación basada en prioridades

Primero en llegar primero en ejecutar FCFS

- *Run queue* gestionada como cola FIFO
- Algoritmo simple que optimiza el uso de CPU

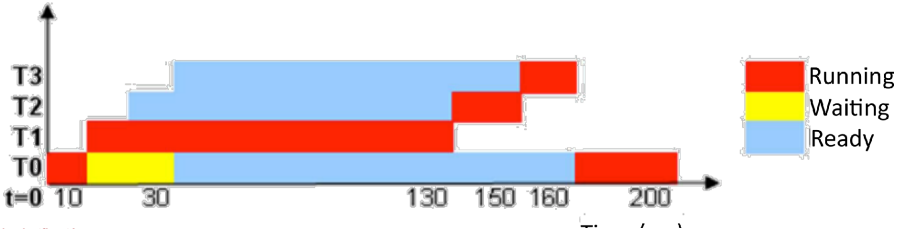
Proceso o thread	Tiempo llegada (ms)	Tiempo CPU (ms)
T0	0	50
T1	10	120
T2	20	20
T3	30	10



Primero en llegar primero en ejecutar FCFS

- Programas con E/S son encolados al final
- Programas largos afectan al sistema

Proceso o thread	Tiempo llegada (ms)	Tiempo CPU (ms)
T0	0	50
T1	10	120
T2	20	20
T3	30	10



Primero el trabajo más corto SJF

- Bueno para programas interactivos
- Necesita conocer el perfil de las tareas
- Problemas de inanición

Proceso o thread	Tiempo llegada (ms)	Tiempo CPU (ms)
T0	0	50
T1	10	120
T2	20	20
T3	30	10



Planificación basada en prioridades

- El usuario especifica el nivel de urgencia de cada proceso
- Problema de inanición:
  - Solución: Aumento de la prioridad con la edad

Proceso o thread	T. llegada (ms)	T. CPU (ms)	Prioridad
T0	0	50	4
T1	10	120	3
T2	20	20	1
T3	30	10	2



Algoritmos expropriativos

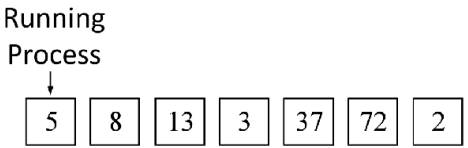
- No expropiativos no son adecuados para SSOO de propósito general
  - Mezcla de trabajos interactivos y trabajos intensivos en CPU
- Los algoritmos expropiativos se activan periódicamente
  - El temporizador del sistema se configura para generar interrupciones periódicas por cada CPU (~ms)
  - Cada interrupción se denomina *tick*
  - Config. por defecto en Linux/x86: 250 ticks por segundo (4ms)

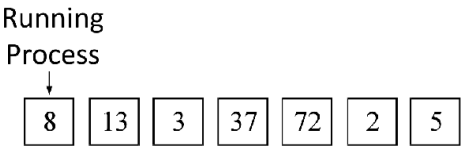
Algoritmos

- Round Robin - RR (turno rotatorio)
- Primero el de menor tiempo restante - SRTF
  - Shortest Remaining Time First
- Prioridad expropiativa
- Colas multinivel

Round Robin - RR (I)

- La planificación se realiza dividiendo el tiempo de CPU en rodajas llamadas *quanto* o *time slice* (expresado en ticks)
- RR: -> FCFS + time slice
  - El planificador expropia al proceso en ejecución cuando consume su *time slice*
  - Cuando proceso es expropiado, RR lo inserta al final de la cola
  - Implementación: cada proceso tiene un contador de *ticks* asociado
  - Inicialmente contador= *ticks time slice*
  - Cada tick, el planificador decrementa el contador del proceso/hilo en ejecución
  - Expropiación <-> contador = 0

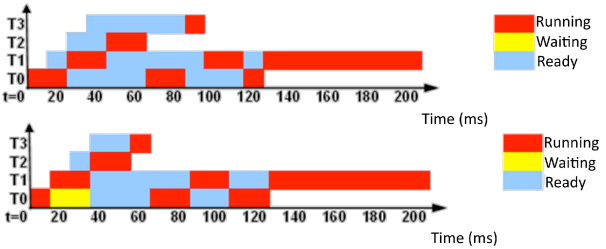




Round Robin - RR (I)

- Timeslice = 20ms

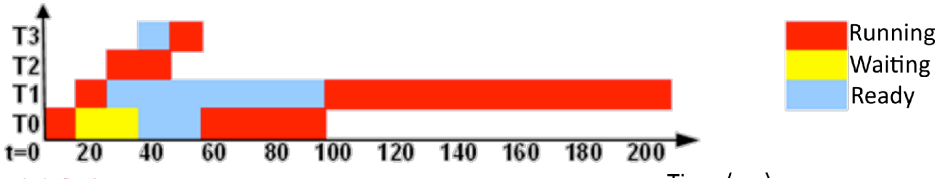
Proceso o thread	Tiempo llegada (ms)	Tiempo CPU (ms)
T0	0	50
T1	10	120
T2	20	20
T3	30	10



Primero el de menor tiempo restante (SRTF)

- SRTF: SJF + expropiación
  - Bueno para programas interactivos
  - Necesita conocer el perfil de las tareas
  - Problemas de inanición

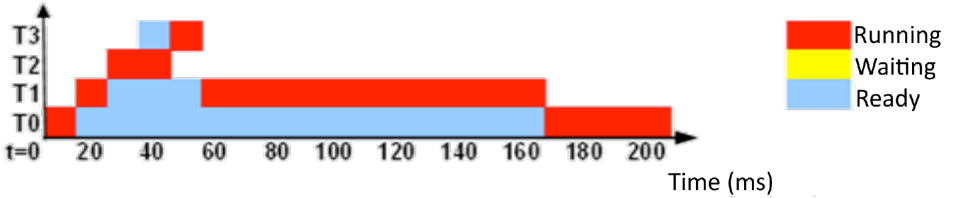
Proceso o thread	Tiempo llegada (ms)	Tiempo CPU (ms)
T0	0	50
T1	10	120
T2	20	20
T3	30	10



Expropiativo basado en prioridades

- El usuario especifica el nivel de urgencia de cada proceso
- Problema de inanición:
  - Solución: Aumento de la prioridad con la edad

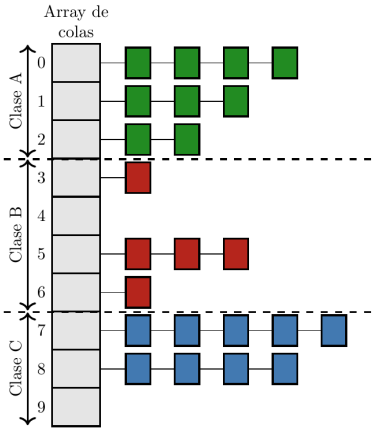
Proceso o thread	T. llegada (ms)	T. CPU (ms)	Prioridad
T0	0	50	4
T1	10	120	3
T2	20	20	1
T3	30	10	2



Planificación con colas multinivel (I)

- Objetivo: dar soporte a distintas clases de procesos
- En el sistema existen k niveles de prioridad
  - Se mantiene una cola de procesos para cada nivel (array de colas)
  - En cada nivel de prioridad puede haber un *time slice* diferente
- Los niveles de prioridad se agrupan en rangos para dar servicio a distintos tipos de proceso-s/hilos
  - Tiempo real
  - Hilos de sistema
  - Interactivos
  - Batch
  - ...

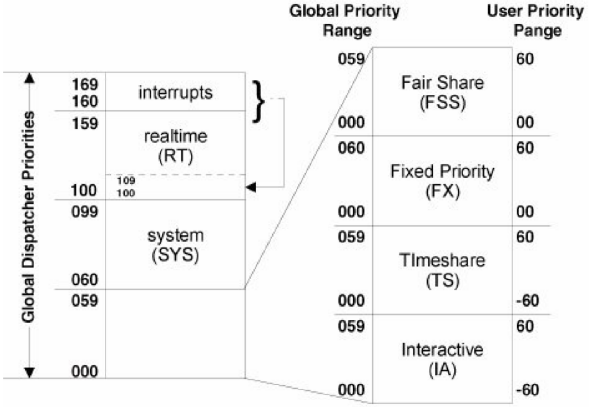
Planificación con colas multinivel (II)



Planificación con colas multinivel (III)

- La planificación se realiza a dos niveles:
  - Global (*dispatcher*): Elección del siguiente proceso a ejecutar y realización de cambios de contexto
    - Siempre se escoge el proceso más prioritario del sistema que está listo para ejecutar
  - Local (*scheduling class*): Gestión las colas asociadas a un determinado rango de prioridades (tipo particular de procesos)
    - Gestión de timeslices y procesamiento de tick
    - La clase de planificación *decide cuándo se ha de expropiar al proceso actual*
    - Invocación al dispatcher para efectuar expropiaciones de usuario

Planificador de Solaris

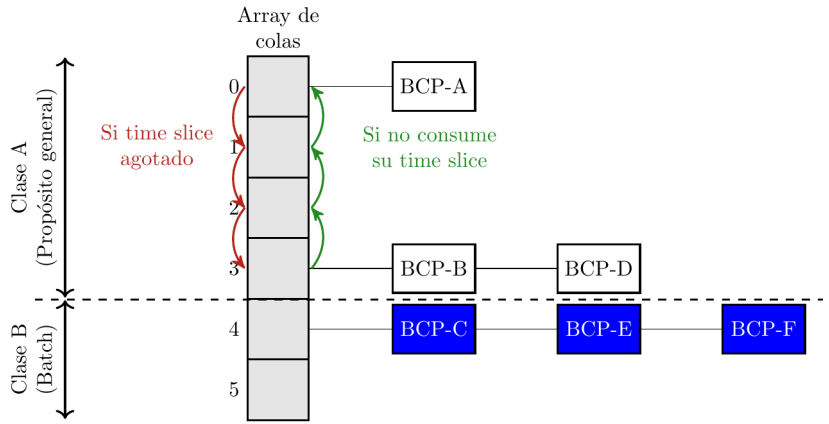


Planificación con colas multinivel (IV)

Dos alternativas de gestión de las colas de procesos

1. Sin realimentación (procesos con prioridad fija)
   - Proceso en la misma cola (cuando está listo para ejecutar)
2. Con realimentación (procesos con prioridad dinámica)
   - Los procesos pueden cambiar de nivel
   - El cambio de nivel sólo se produce dentro del rango de prioridades gestionado por el "planificador local"
   - Necesario definir política de cambio de nivel
   - Ejemplo: Política para favorecer a procesos interactivos
     - Si proceso agota su timeslice, baja de nivel
     - Si proceso no agota su timeslice (p. ej., bloqueo E/S), sube de nivel

Ejemplo: Multinivel con realimentación



Cola multinivel realimentada: motivación

Motivación y objetivos

- Diferentes tipos de cargas de trabajo: interactivas, intensivas en CPU, intensivas en E/S, ...
- Ninguno de los algoritmos anteriores se adapta bien a todas
- Idea: combinar varios algoritmos, cada uno ajustado a un perfil
  - [Objetivo 1] Turnaround: comportarse como SJF para trabajos intensivos en CPU
  - [Objetivo 2] Tiempo de respuesta: comportarse como RR para trabajos interactivos
  - Estos dos objetivos pueden ser contradictorios

Retos

- El tipo de trabajo o su patrón futuro de uso de CPU no se conoce de antemano.
- El esquema debe, aun así, permitir a un administrador especificar la prioridad relativa de un trabajo.

Cola multinivel realimentada: estructura

- Reglas
  1. Los trabajos en una cola de mayor prioridad se ejecutan antes que los de una cola de menor prioridad.
  2. Los trabajos que comparten el mismo nivel de prioridad se ejecutan entre sí mediante RR.
  3. Un trabajo nuevo siempre empieza en la cola de mayor prioridad.
  4. Si un trabajo agota todo su intervalo de tiempo, se le reduce la prioridad (baja una cola, $Q_i \rightarrow Q_{i+1}$)

Cola multinivel realimentada: ejemplo resuelto

Ejemplo 1: trabajo interactivo protegido

- Un trabajo largo intensivo en CPU comparte el sistema con un trabajo interactivo
- Las ráfagas de CPU del trabajo interactivo son más cortas que el intervalo de tiempo, así que nunca agota su cuanto y mantiene su alta prioridad
- El trabajo intensivo en CPU consume repetidamente todo su intervalo de tiempo y va bajando hasta la cola de menor prioridad
- Resultado: buen tiempo de respuesta para el trabajo interactivo, buen turnaround para el trabajo intensivo en CPU

Cola multinivel realimentada: ejemplo resuelto

Ejemplo 2: inanición

- Un trabajo largo intensivo en CPU comparte el sistema con un flujo constante de trabajos cortos que llegan uno tras otro
- El suministro constante de trabajos cortos de alta prioridad puede impedir que el trabajo intensivo en CPU llegue a planificarse nunca
- Resultado: inanición del trabajo intensivo en CPU

Cola multinivel realimentada

Problema: inanición

- Los trabajos intensivos en CPU (empujados a colas de baja prioridad) pueden sufrir inanición por culpa de un gran número de trabajos interactivos (alta prioridad)
- **Solución - Priority Boost** : elevar periódicamente la prioridad de todas las tareas (devolver todas las tareas a la cola de mayor prioridad)

Cola multinivel realimentada

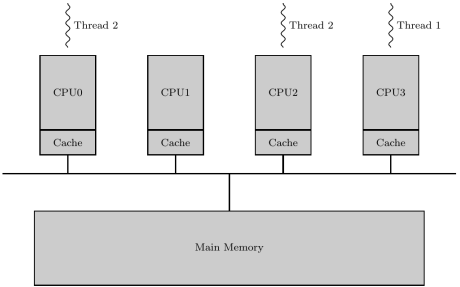
Problema: engañar al planificador

- Un trabajo que cede voluntariamente la CPU justo antes de que expire su intervalo de tiempo no pierde prioridad, puede seguir reclamando la CPU mientras se comporta como si fuera intensivo en CPU
- **Solución - Contabilidad de CPU** : llevar la cuenta del tiempo *total* de CPU consumido en el nivel Q_i; en cuanto ese total supera el intervalo de tiempo, se reduce la prioridad, independientemente de cómo se repartió el tiempo de CPU entre varias ráfagas
- Nota: las tareas nunca suben de nivel por sí solas. Una tarea que se vuelve interactiva brevemente tras una fase intensiva en CPU permanece en la prioridad más baja hasta el siguiente Priority Boost

Planificación SMP

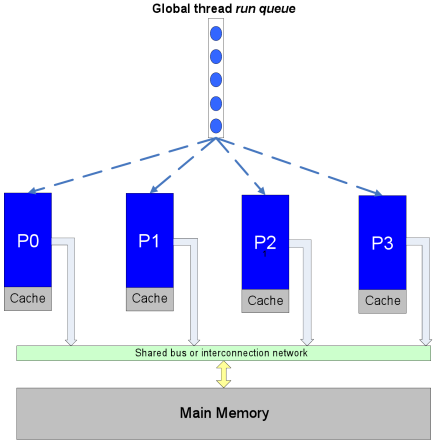
- SMP (*Symmetric Multi-Processing*)
  - Garantizar un equilibrio de carga (load balancing)
    - Que no haya un procesador ocioso y otros con mucha carga de trabajo
  - Tener en cuenta la afinidad de procesos y procesadores
    - Importante al replanificar un proceso
    - Evitar realizar migraciones de hilos
- Tener en cuenta la compartición de datos entre procesos/hilos si hay varios nodos de memoria (NUMA)
  - Si dos hilos comparten memoria, probablemente sea bueno que compartan todo lo posible su nivel de jerarquía

Concepto de afinidad



Un nuevo hilo entra al sistema (Thread 2). Al ejecutarse, carga parte de sus datos en la cache (CPU0). El hilo desarrolla afinidad a la CPU0 (cache hot). El hilo 2 se bloquea por E/S. El hilo 2 se despierta (fin E/S) y el planificador lo asigna a un procesador diferente (CPU 2). Migración de hilo → degradación del rendimiento.

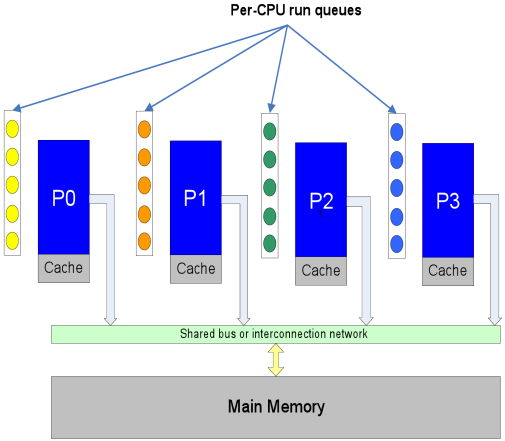
Planificación SMP (Linux v2.4.x)



Planificación SMP (Linux v2.4.x)

- Una única run queue para todos los procesadores
- Equilibrio en la carga
  - Todos los procesadores tienen potencialmente el mismo trabajo
- Malo para la afinidad
  - El proceso A se ejecutó en la CPU1 y luego se envía a CPU2 (migración)
  - Migración $\rightarrow$ reconstruir estado de cache
- Problemas de escalabilidad

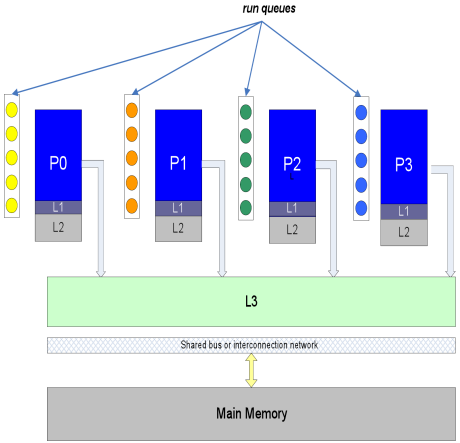
Planificación SMP (Linux v2.6.x+)



Planificación SMP (Linux v2.6.x+)

- Una cola de ejecución por procesador
- Mayor escalabilidad
- Periódicamente (o bajo demanda) se ejecuta el equilibrador de carga
  - Considera qué procesos pueden/deben migrarse
  - Tiene en cuenta la afinidad
- Este modelo es el que utilizan la mayor parte de SSOO actuales de propósito general (Linux, Solaris, FreeBSD o MS Windows)

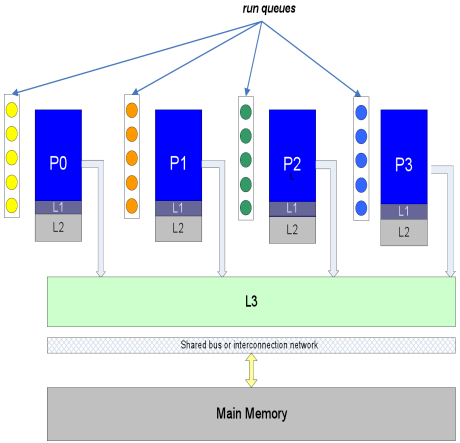
Planificación en multicore



Planificación en multicore

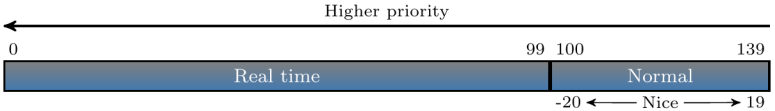
- El SO ve cada core como un procesador independiente, pero no lo es
  - Algún nivel de cache compartido entre cores
- Potencial degradación del rendimiento por contención en recursos compartidos
- Problemas de justicia
- Planificación en multicore: área de investigación activa

Planificación en Linux



Planificación en Linux (v6.6+)

- **140 niveles de prioridad**
  - 100 para procesos en tiempo real
    - 3 políticas de planificación: *deadline*, RR, y FIFO
  - 40 para procesos normales (Fair)
    - Por defecto: algoritmo EEVDF (*Earliest Eligible Virtual Deadline First*)
    - Es posible "inyectar" una política de planificación desarrollada con eBPF mediante la clase `sched-ext`
    - La prioridad puede cambiar con el comando nice
    - ej: `$ nice -n <nice_value> <application_command>` y `<nice_value>` está [-20, 19]



Políticas de planificación en Linux

SCHED_OTHER

Política de reparto de tiempo estándar, prioridad 0, usa el valor *nice* (entre -20 y 19, por defecto 0)* para repartir la CPU. * Implementada por el Completely Fair Scheduler (CFS) hasta Linux 6.6, y por EEVDF a partir de 6.8.

Políticas de tiempo real

- `SCHED_FIFO`: política FIFO de tiempo real con prioridades 1-99; una tarea se ejecuta hasta que se bloquea por E/S, es desalojada por una tarea de mayor prioridad, o cede el control voluntariamente
- `SCHED_RR`: igual que SCHED*FIFO , pero las tareas que comparten prioridad se ejecutan en turnos round-robin (100 ms por defecto)
- Ver `sched(7)` para la referencia completa

Earliest Eligible Virtual Deadline First (EEVDF)

- Incorporado en el kernel de Linux desde la v6.8, aunque propuesto en 1995
  - I. Stoica and H. Abdel-Wahab. 1995, *"Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation"*. Technical Report. Old Dominion University, USA.

Objetivos de EEVDF

- Garantizar un reparto justo del tiempo de CPU teniendo en cuenta la prioridad del proceso
  - *Planificador de reparto proporcional*
- Soportar distintos tipos de procesos: multimedia (tiempo real flexible), interactivos, por lotes, etc.
- Proporcionar buenos tiempos de respuesta
  - Adecuado para entornos interactivos (GUIs)

Planificador de reparto proporcional



- Si 4 hilos de la misma prioridad estuvieran en el sistema durante 40 ms, cada hilo debería ejecutarse durante 10 ms para garantizar un reparto uniforme (*fairness*)
- *¿Y si los hilos tienen distintas prioridades?*
  - EEVDF asigna a cada proceso un peso, directamente proporcional a la prioridad

Asignación de tiempo de CPU en EEVDF

- Considérese un intervalo de ejecución arbitrario (*sched period*) con n procesos activos P1, P2, ... , Pn
- Asignación de tiempo de CPU proporcional al peso (w_i) de cada proceso

$$T_{fair}(P) = sched_period_ms \cdot \frac{w_P}{\sum_{i=1}^{n} w_i}$$

- Ejemplo:
  - 3 procesos A, B y C, con pesos 2, 2 y 1, respectivamente
  - `sched_period_ms=20ms`
  - $T_{fair}(A) = T_{fair}(B) = 8ms$ y $T_{f}air(C) = 4ms$

Reparto proporcional de CPU con EEVDF

- ¿Cómo reducir la sobrecarga de los cambios de contexto al repartir la CPU?
  - EEVDF usa un cuanto fijo
  - La selección del siguiente proceso ocurre en cada cuanto o cuando la CPU está inactiva
- ¿Cómo repartir la CPU en escenarios muy dinámicos donde los procesos se bloquean/activan con frecuencia?
  - EEVDF usa la noción de **tiempo virtual** , que avanza más lentamente cuantos más procesos estén activos y más pesados sean

$$VirtualTimeUnit = 1/\sum_{i=1}^{n} w_i$$

- Cuando un proceso entra o sale del sistema, el tiempo virtual se ajusta para garantizar el reparto justo

EEVDF: tiempo de elegibilidad y plazo virtual

- El planificador mantiene un contador de tiempo virtual V(T) para cada CPU
  - Se incrementa cuando un proceso consume un *tick* o una fracción (se bloquea)

$$V(T) += T / \sum_{i=1}^{n} w_i$$

- Cada proceso tiene un tiempo de servicio esperado T S
  - Si no lo proporciona el usuario, se asume un valor por defecto

EEVDF asigna dos valores a cada proceso

- ve: *Tiempo Virtual de Elegibilidad*
  - Denota la marca de tiempo virtual a partir de la cual el proceso puede ser seleccionado para ejecutarse
  - Elegible si $Ve \leq V(T)$
- vd: Plazo virtual
  - Marca de tiempo virtual que expresa la urgencia de ejecución

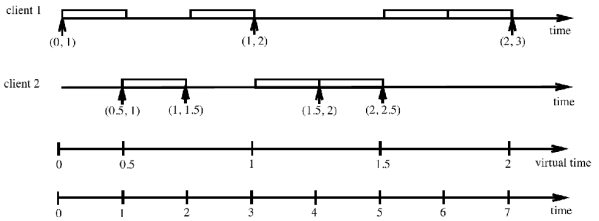
EEVDF: tiempo de elegibilidad y plazo virtual

- EEVDF selecciona el proceso elegible (con $ve \leq T_{virtual}$) con el menor plazo virtual (vd)

Actualización de ve y vd

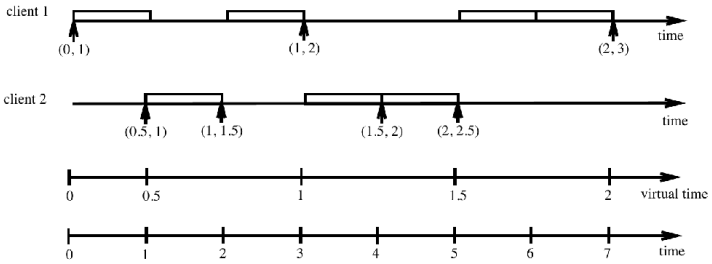
- Cuando un proceso entra en el sistema
  - $ve = V(T)$
  - $vd = ve + (T_S / w_P)$
- Si el proceso completa T_S, el nuevo ve es el plazo anterior
  - $ve = vd$
- Si el proceso no completa T_S (se bloquea), entonces vd se ajusta proporcionalmente
  - $ve = ve + (T_{consumed})/(w_P)$

EEVDF: Ejemplo



Proceso/th	T. llegada	w_i	T_s (quanto)
client 1	0	2	2
client 2	1	2	1

EEVDF: ejemplo



- En el ejemplo $(ve, vd) = (V(T)_{ready}, V(T)_{ready} + (T_S/w_P))$
- Para $T \in [0, 1) \Rightarrow \Delta V(T) = 0.5$
- Para $T \geq 1 \Rightarrow \Delta V(T) = 0.25$

EEVDF: gestión de escenarios dinámicos

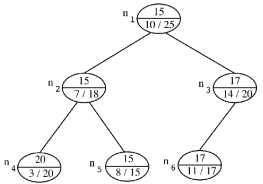
- ¿Qué ocurre si un proceso consume más o menos tiempo de CPU del que le corresponde y se bloquea?
- EEVDF introduce el concepto de lag
  - Sea t el intervalo de tiempo arbitrario durante el cual el proceso P está activo
  - $Lag_P(t) = T_{fair,P}(t) - T_{consumed,P}(t)$
  - Si $Lag_P(t) < 0$ el proceso ha consumido más de lo que le corresponde
- El tiempo virtual se ajusta teniendo en cuenta el conjunto de procesos activos A y el lag del proceso entrante/saliente:
  - El proceso j sale:
    - $V(T) = V(T) + \frac{lag_j}{\sum_{i \in A} w_i}$
  - El proceso j entra:
    - $V(T) = V(T) - \frac{lag_j}{\sum_{i \in A} w_i}$

EEVDF: gestión de escenarios dinámicos

- ¿Qué ocurre si el usuario altera la prioridad (peso) de un proceso en ejecución?
  - Equivale a que salga un proceso de peso wi (antiguo) y entre otro con peso w'j (nuevo valor)
- Ajuste asociado del tiempo virtual V(T):
  - $V(T) = V(T) + \frac{lag_j}{(\sum_{i \in A} w_i) - w_j} - \frac{lag_j}{(\sum_{i \in A} w_i) - w_j + w_j'}$

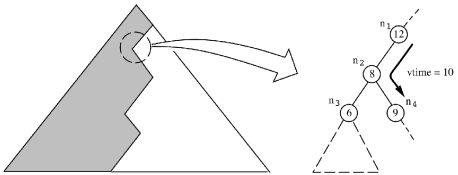
EEVDF: selección del siguiente proceso

- EEVDF mantiene una "lista" de procesos (por cada CPU) ordenada por ve de forma ascendente
  - Una run queue por CPU, formada por los BCP de los procesos asignados a esa CPU (posiblemente con distintas prioridades)
- Por razones de eficiencia, se usa un árbol rojo-negro para implementar la "lista":
  - Árbol equilibrado: operaciones $O(logN)$
- Cada nodo almacena el ve y el vd del proceso, así como min_vd
  - *min_vd*: Plazo virtual mínimo en el subárbol asociado a ese nodo
- La clave del nodo es *ve*



EEVDF: selección del siguiente proceso

- Para seleccionar el siguiente proceso, EEVDF divide el árbol equilibrado en dos subárboles
  - Nodos elegibles (ve $\leq$ V(T))
  - Nodos no elegibles (ve $>$ V(T))
- El siguiente proceso -> El que tenga el menor vd entre los nodos elegibles



EEVDF: uniendo todas las piezas

Esquema del algoritmo

- El planificador mantiene un contador de tiempo virtual global (por CPU) V(T)
  - Avanza más lentamente que el tiempo real, en función del número de procesos activos y su peso
  - Se incrementa a medida que el proceso actual consume tiempo de CPU
- EEVDF asigna a cada proceso un tiempo de elegibilidad (ve) y un plazo virtual (vd)
  - *vd* denota la urgencia o prioridad dinámica del proceso
  - Cuanto más cercano esté el plazo, mayor será el peso y más corta la ráfaga de CPU
- Se selecciona para ejecutar el proceso elegible ($ve \leq V(T)$) con el menor vd
  - Selección eficiente mediante un árbol rojo-negro
  - V(T) se ajusta cuando los procesos entran o salen del sistema, en función del lag del proceso entrante/saliente